SAFE/UFRJ - EVOLUÇÃO ...

Relatório Parcial das atividades do Programa Institucional de Bolsas de Iniciação Científica e Voluntário (PIBIC) Edital 2024

Aluno
André Vinicius Lobo Giron
Engenharia de Controle e Automação
Programa PIBIC
Período de vigência da bolsa: abril de 2024 até julho de 2024

Orientador
Guilherme Horta Travassos
Programa de Engenharia de Sistemas e Computação (PESC)

9 de Julho de 2024

Resumo

A pandemia de COVID-19 colocou em evidência a importância do controle de biossegurança em edifícios, para conter o contágio por agentes biológicos, bem como exigiu a criação de novas soluções para lidar com estes fatores. Por isto, o projeto SAFE-UFRJ pretende desenvolver um dispositivo IoT de baixo custo para realizar o monitoramento destes indicadores e facilitar o controle e manutenção dos espaços, através de um sistema para o gerenciamento de instalações, que permita a visualização do estado de cada sala que possua um dispositivo, tal como possibilite ao usuário requisitar a manutenção do ambiente.

Sumário

1 Introdução **3**

2 Objetivos do Projeto **4**

3 Desenvolvimento **4**

3.1 Evolução do Módulo de Contagem de Pessoas 4

3.2 Análise dos Dados Coletados 7

4 Trabalhos Futuros **11**

Apêndices **12**

1 Introdução

Desde a pandemia de COVID-19, a biossegurança em ambientes fechados passou a receber maior atenção. Aspectos como a qualidade do ar, a densidade de ocupação e a limpeza das instalações são essenciais para a preservação da saúde dos indivíduos que frequentam esses espaços. Entretanto, a preocupação com a qualidade do ar ainda se concentra, em grande parte, no ambiente externo, desconsiderando o fato de que as pessoas passam aproximadamente 90% do seu tempo em ambientes internos [1]. Nesse contexto, a qualidade do ar interior (do inglês Indoor Air Quality – IAQ) assume papel central no bem-estar humano e pode representar um risco significativo à saúde, uma vez que poluentes, vírus, bactérias, fungos e compostos orgânicos voláteis (VOCs) tendem a se acumular com maior intensidade em espaços fechados.

Sintomas como dores de cabeça, fadiga, cansaço excessivo e irritação nos olhos e pele estão entre os malefícios de um ambiente com baixa IAQ [2]. A qualidade do ar em um ambiente interno pode ser agravada devido a diversos fatores, como má circulação de ar, emissores de compostos voláteis indesejados/despercebidos e falta de manutenção do espaço. Além disso, como destacado anteriormente, indivíduos contemporâneos tendem a passar grande parte do seu dia em espaços internos. Dessa forma torna-se essencial monitorar a qualidade do ar nestes ambientes para garantir o bem-estar de seus ocupantes.

Nesse contexto, aplicações de software contemporâneas no paradigma da Internet das Coisas (*Internet of Things* - IoT) e Inteligência Artificial (IA), têm sido grandes aliados para monitorar ambientes internos e identificar situações críticas de qualidade de ar [3]. Embora essas aplicações tenham surgido inicialmente em reação à pandemia de COVID-19, seu potencial vai além, pois podem ser adaptadas para diferentes cenários de emergências e desastres, oferecendo uma abordagem preventiva contra novas epidemias e pandemias [4, 2, 5].

Por isso, foi desenvolvido o sistema SAFE/UFRJ, que permite monitorar as condições de biossegurança [6] em instalações (laboratórios, salas de aula e ambientes de convivência) por meio de dispositivos IoT [7]. Estes dispositivos permitem a coleta de dados para indicar o número de pessoas em determinado ambiente, nível de IAQ, umidade e temperatura da instalação, apresentando essas informações em um *dashboard*. O sistema ainda emite alertas quando níveis de risco (estabelecidos para cada instalação) são atingidos e possibilita o gerenciamento das instalações cadastradas, incluindo a solicitação de serviços de limpeza ou manutenção conforme o necessário [2].

O sistema foi avaliado e evoluído com base nos requisitos elicitados em reuniões com os grupos de pesquisadores que compõem o projeto, sendo eles: desenvolvedores, analistas de requisitos e gerência. Desta forma, o sistema em seu estágio de desenvolvimento atual evidencia seu potencial no monitoramento de ambientes internos, tendo em vista que consegue coletar e apresentar informações acerca dos bioindicadores de espaços que possuem um dispositivo BIoT instalado, tudo isso, com uma periodicidade útil para auxiliar na tomada de decisões acerca destes espaços.

O presente trabalho pretende descrever dois feitos principais: i) a evolução do módulo de contagem de pessoas; ii) desenvolvimento de testes e abordagens para avaliar a correspondência dos dados coletados pelo dispositivo BIoT com os valores apresentados pelo sistema.

Portanto, os trabalhos realizados durante o período de vigência da bolsa PIBIC, compreenderam

a evolução do projeto já criado e em desenvolvimento, tendo ênfase na evolução do dispositivo IoT e seu funcionamento, bem como na manutenção e expansão das funcionalidades do *backend* e *dashboard* do sistema.

2 Objetivos do Projeto

Os objetivos do presente trabalho consistem em: i) evoluir o módulo de contagem de pessoas presente no dispositivo BIoT em conformidade com os requisitos previstos na documentação do mesmo; ii) construção de um novo dispositivo, para se observar a compatibilidade de valores coletados, bem como a semelhança no comportamento das séries históricas armazenadas.

3 Desenvolvimento

A presente seção descreve de forma sistemática os processos realizados e os esforços empreendidos com vistas ao alcance dos objetivos previamente estabelecidos. Para assegurar a clareza da exposição e preservar o contexto de cada etapa do trabalho, o conteúdo foi organizado nas Subseções 3.1 e 3.2, de modo a facilitar a compreensão do leitor.

3.1 Evolução do Módulo de Contagem de Pessoas

Assim como descrito anteriormente, o módulo de contagem de pessoas do dispositivo BIoT configura os sensores utilizados para coleta dos dados, bem como a lógica empregada na aquisição e processamento das informações. Tendo como principal objetivo, comunicar ao sistema a entrada ou saída de qualquer indivíduo em uma instalação que possua um dispositivo BIoT. Comumente, sistemas de software IoT baseiam-se na coleta periódica de informações, como forma de realizar o monitoramento, isto resulta na obtenção de séries temporais que correspondem ao comportamento do fenômeno em observação, neste caso, a entrada e saída de pessoas em uma instalação.

O módulo de sensoreamento para contagem de pessoas, é composto por dois sensores ultrassônicos HC-SR04, que possuem invólucros e estrutura construída através de impressão 3D. Estes sensores ficam posicionados acima da porta da instalação onde deseja-se contabilizar o número de pessoas, assim como apresentado na Figura 1. Os sensores ultrassônicos são capazes de coletar medidas acerca da distância entre eles e indivíduos ou objetos que estejam abaixo deles e no raio de alcance do seu emissor e receptor de ondas, possibilitando o sensoreamento da área de entrada/saída do ambiente escolhido.

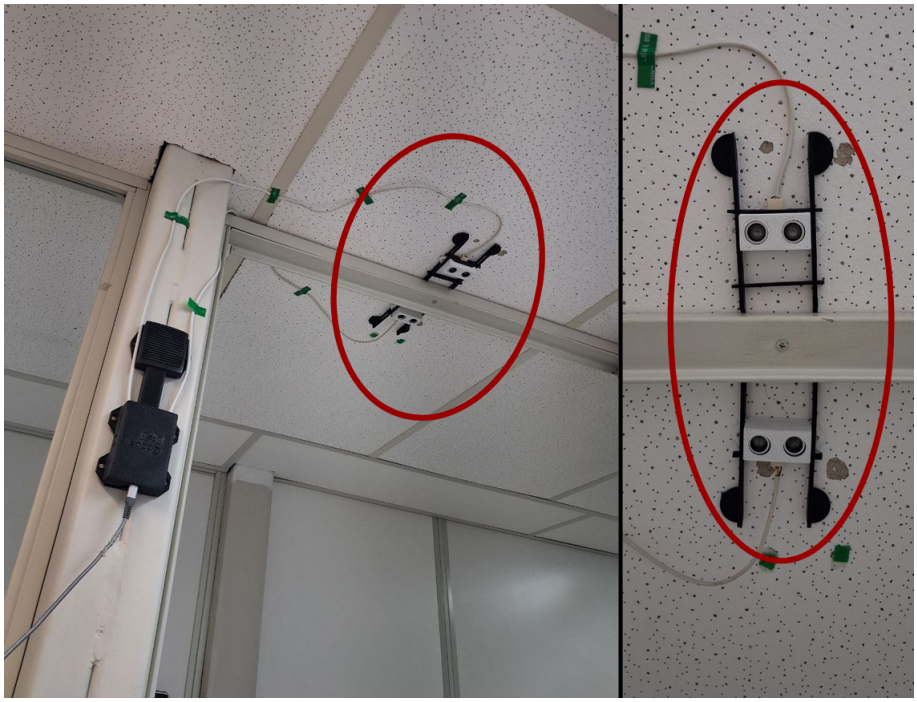


Figura 1: Módulo de Sensoreamento para Contagem de Pessoas.

O primeiro algoritmo proposto, para monitorar a quantidade de indivíduos em uma instalação baseia-se principalmente na disposição dos sensores ultrassônicos, que pode ser observado na Figura 1, e no caráter contínuo dos dados coletados pelo dispositivo BIoT. Por este motivo, para cada evento de entrada/saída, a série temporal coletada possui padrões bem definidos de ativação dos sensores. No caso de uma entrada, o sensor externo será ativado antes do sensor interno, bem como será o primeiro a ser desativado, para eventos de saída, o contrário acontece. Deste modo, o rastreamento de eventos de interesse, torna-se uma tarefa de identificação de padrões de ativação dos sensores através dos dados coletados.

Para lidar com este objetivo, o algortimo construído consiste primeiramente na binarização dos dados coletados através de um limite pré-estabelecido, para que os estados dos sensores (ativado/desativado) sejam facilmente diferenciados, assim como descrito no Algoritmo 1. Após a binarização, o algoritmo compara o estado atual(tupla com dois valores booleanos) com o estado anterior, para identificar e armazenar toda mudança de estado. Sempre que ocorre uma mudança no estado, a tupla correspondente a este estado é armazenada em um vetor que atua como janela deslizante, após ser armazenado o último estado do vetor é descartado.

está descrita no Algoritmo 1

A lógica implementada para identificar e contabilizar estes eventos de interesse está descrita no Algoritmo 1.

faz uso destas características e esta na presente no Algoritmo 1.

que durante os eventos de interesse, ou seja, durante a entrada ou saída de um indivíduo,

Algoritmo 1 Lógica Inicial Para Contagem de Pessoas

```
function COMPARAR_VETORES(A, B)
                                                    ▷ A e B são vetores
    if |A| ≠ |B| then                ▷ Compara se os vetores possuem o mesmo tamanho
        return false
    end if
    for i ← 1 to |A| do                ▷ Compara o valor de cada item dos vetores
        if A[i] ≠ B[i] then
            return false
        end if
    end for
    return true                        ▷ Verdadeiro caso os vetores sejam iguais
end function

function CALCULAR_NUMERO_DE_PESSOAS(M)
                                                    ▷ M é o vetor de medidas brutas
    b ← (0, 0)                       ▷ Vetor que armazena o estado atual binarizado
    entrada ← 0
    saida ← 0
    TOLERANCIA ← 60                                  ▷ Limite para a binarização
    W ← [(0, 0), (0, 0), (0, 0), (0, 0), (0, 0)]     ▷ Vetor para o espaço de estados
    for each m ∈ M do
                                        ▷ — Etapa de binarização das medidas brutas—
        if m.distancia_sensor_interno < TOLERANCIA then
            b_1 ← 1
        else
            b_1 ← 0
        end if
        if m.distancia_sensor_externo < TOLERANCIA then
            b_2 ← 1
        else
            b_2 ← 0
        end if
                                    ▷ — Etapa de construção do padrão de bits capturado —
        if not COMPARAR_VETORES(b, W[5]) then
            SHIFT_LEFT(W)                                  ▷ Descarta estado antigo
            W[5] ← b                                       ▷ Armazena o novo estado
                                    ▷ — Etapa de identificação do evento de interesse —
            if COMPARAR_VETORES(W, PADRAO_DE_ENTRADA) then
                entrada ← entrada + 1
            else if COMPARAR_VETORES(W, PADRAO_DE_SAIDA) then
                saida ← saida + 1
            end if
                                                ▷ Padrões inválidos não geram contagem
        end if
    end for
    return (entrada, saida)
end function
```

3.2 Análise dos Dados Coletados

Os trabalhos desenvolvidos partem do contexto já existente do projeto, tendo em vista o nível de maturidade do sistema, bem como todas as decisões arquiteturais anteriores. Sendo assim, esta seção compreende as melhorias implementadas acerca do projeto já proposto, o qual é apresentado na Figura 1.

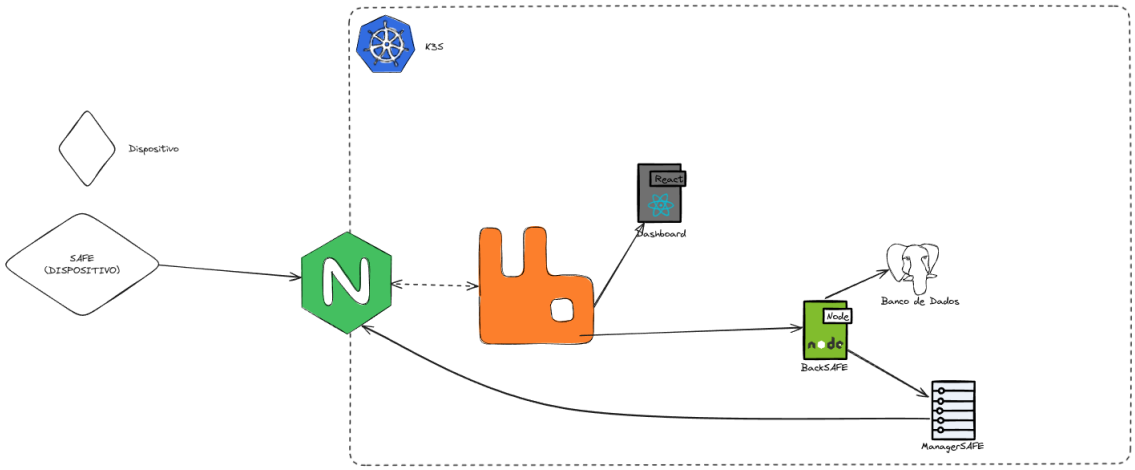


Figura 2: Esquemático simplificado do projeto SAFE-UFRJ.

O sistema existente, é constituído por um dispositivo IoT que realiza as medições acerca do ambiente e publica as informações via *broker* através do protocolo MQTT(*Message Queuing Telemetry Transport*), a ferramenta utilizada para a comunicação é o RabbitMQ. As especificações do dispositivo, consistem em um ESP32 com FreeRTOS para o escalonamento das tarefas, que faz uso dos sensores CCS811(dióxido de carbono e compostos voláteis), DHT11(temperatura e umidade) e dois HC-SR04(controle do fluxo de pessoas), para realização das medidas.

Ademais, o *backend* consome estes tópicos publicados pelo dispositivo, transforma as informações em dados mais tangíveis ao usuário e os armazena no banco de dados relacional(PostgreSQL), bem como disponibiliza rotas de uma API(*Application Programming Interface*) para o funcionamento da interface de gerenciamento de usuários e instalações do projeto SAFE-UFRJ(*ManagerSAFE*). Foram utilizados respectivamente Typescript e Express para a construção do *backend*, bem como o *dashboard* foi desenvolvido em Javascript.

Assim, um dos primeiros trabalhos desenvolvidos como bolsista consistiu-se na evolução do código do dispositivo de acordo com o diagrama presente no Apêndice A, para garantir melhor sincronicidade entre as tarefas, tendo em vista que para sistemas operacionais em tempo real, como o FreeRTOS, este é um fator crucial para o funcionamento.

Também, foram implementados classificadores de erro, para identificar medidas que foram realizadas com insucesso ou que aconteceram durante o mau funcionamento de algum componente do dispositivo, comprometendo a medição. Para isto, foram construídos tipos de erro, como o apresentado na Figura 3, permitindo a distinção entre medições corretas e insucessos.

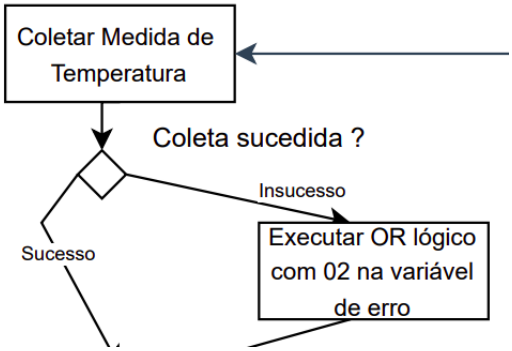


Figura 3: Exemplo de tipo de erro.

O tipo de erro é classificado em 64 *bits* e cada casa em sua representação binária equivale a um tipo de erro único. Assim, caso o dispositivo tenha insucesso apenas na coleta de temperatura, o identificador que deve ser inserido na variável de erro é 0000010. Esta checagem é realizada em todas as etapas da medição, bem como na obtenção do registro de horário da coleta, como presente no diagrama do Apêndice A.

Após isso, a ênfase se voltou para a visualização das informações coletadas. Como apresentado na Figura 2 o *dashboard* coleta informações diretamente do *broker* para apresentá-las ao usuário. No entanto, a exibição das medidas de maneira bruta, assim como são coletadas pelos sensores, não apresenta tanto potencial de tomada de decisão ao usuário.

Desta forma, o próximo trabalho desenvolvido durante a vigência da bolsa PIBIC compreende a implementação de rotas para a API já existente, possibilitando o consumo das medições armazenadas no banco de dados de maneira assíncrona e apresentando informações de maior valor agregado ao usuário via *dashboard*. Nesta etapa, foram desenvolvidas rotas tanto para o consumo das informações à respeito das instalações, bem como a apresentação das medidas relacionadas a cada dispositivo ou instalação, facilitando possíveis testes futuros.

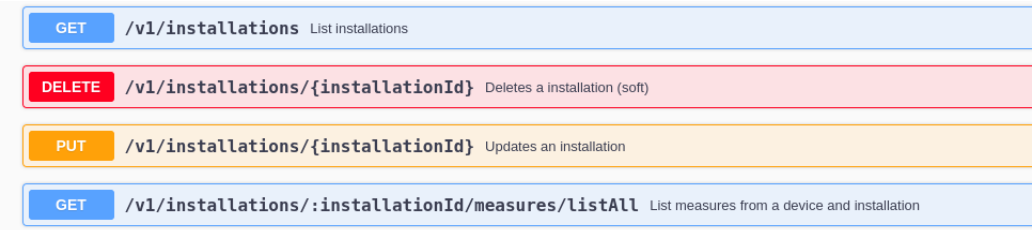


Figura 4: Recorte das rotas presentes na API.

A Figura 4, apresenta um recorte da documentação da API contendo as rotas implementadas, sendo elas encarregadas pela listagem de instalações e a listagem de medidas.

Por fim, a última etapa desenvolvida durante este período, abrange a adequação do *dashboard* do sistema para o consumo das rotas desenvolvidas e apresentação das medições. Para isto, foi utilizada a estratégia de *pooling* para o consumo da API, a qual consiste em realizar consultas periódicas para alimentação dos componentes presentes no *dashboard*, logo as informações presentes na interface são atualizadas a cada 15 segundos.

Desta forma, cada instalação é apresentada como um retângulo, contendo a identificação, localização, medidas de biossegurança e o registro de hora da última medida, assim como apresentado

na Figura 4.

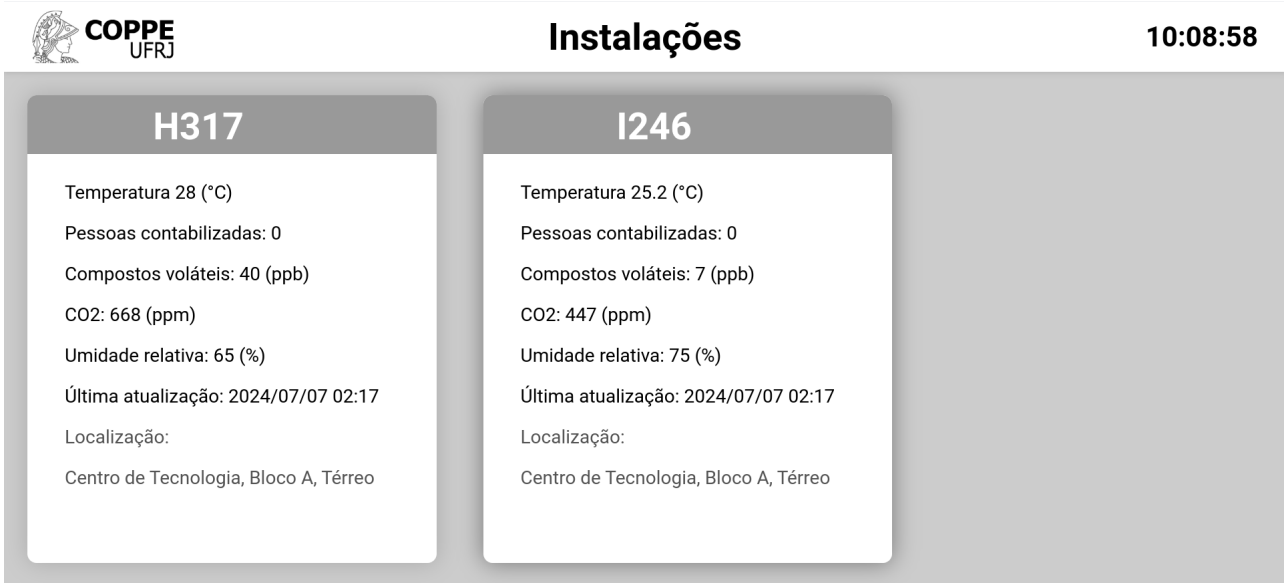


Figura 5: Tela de instalações do *dashboard* do sistema.

As informações apresentadas em cada retângulo, correspondem as medidas coletadas pelo dispositivo presente naquela instalação. Como considera-se que há apenas um dispositivo por sala (instalação), não se faz necessário a adoção de técnicas para unir informações de dois dispositivos.

Ademais, o usuário pode obter mais informações acerca da instalação desejada apenas selecionando o respectivo retângulo. Para isto, foi implementada a tela de Detalhes da Instalação apresentada na Figura 6, a qual apresenta os relatórios de cada propriedade da medição.

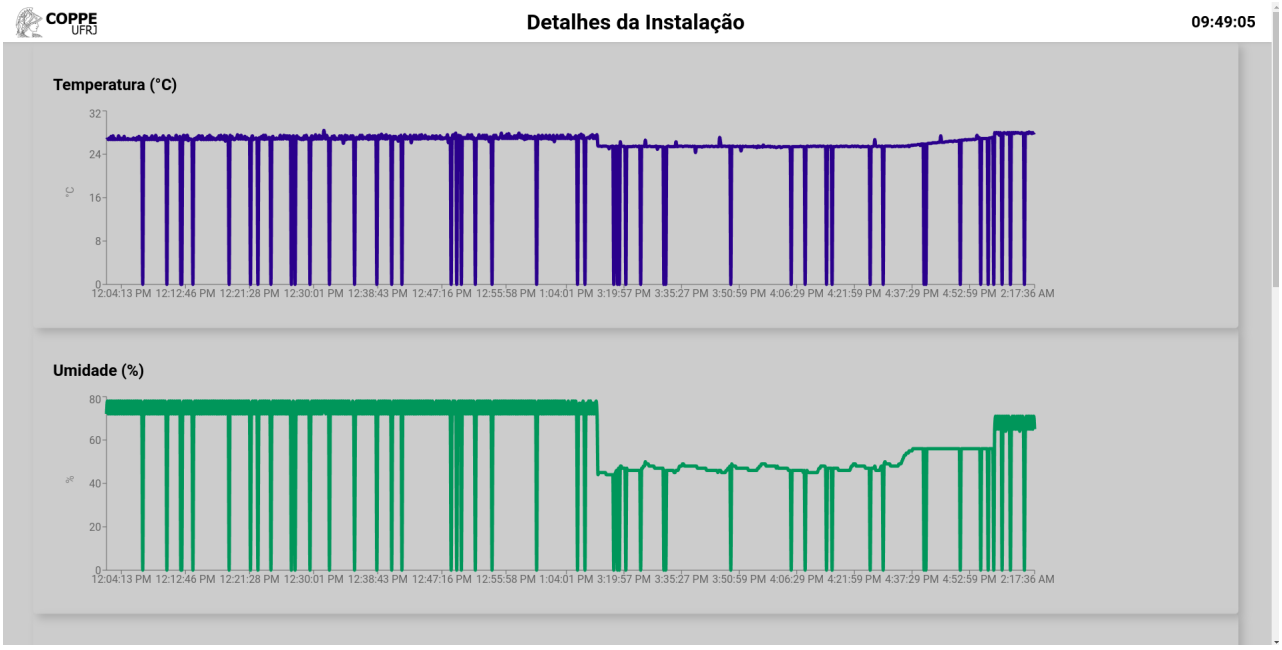


Figura 6: Tela de Detalhes da Instalação.

Os gráficos presentes nesta tela, apresentam as medições realizadas, bem como o registro de horário de cada medição, permitindo ao usuário observar variações e padrões nos indicadores de

cada instalação. As barras verticais presentes nos gráficos, correspondem aos erros de medição descritos anteriormente, caso o dispositivo detecte algum problema durante a realização da medida, o mesmo realiza a publicação do tópico, mas adiciona um tipo de erro ao pacote, bem como altera o valor da medida que foi realizada com insucesso, para um valor fora do escopo de leitura dos sensores e consequentemente invalidando a medida.

4 Trabalhos Futuros

O presente projeto aborda um tema relevante para o contemporâneo em um contexto atrativo para o âmbito da tecnologia. Além disso, apresenta um nível de maturidade médio e bastante robustez no quesito de tecnologias desenvolvidas. Desta forma, os trabalhos futuros para o mesmo consistem em:

1. Ajustar os detalhes da instalação;
2. Tratar erros de medição;
3. Integrar o *dashboard* junto ao *ManagerSAFE*
4. Avaliar o dispositivo;
5. Avaliar o *dashboard*;
6. Avaliar o *ManagerSAFE*;

Além disso, o projeto SAFE faz parte de uma família de dispositivos, logo, após a conclusão do desenvolvimento acerca deste projeto pretende-se retomar o desenvolvimento do OxímetroIoT, implementando atualizações parecidas tendo em vista a arquitetura semelhante entre os projetos, mas adequando-se as particularidades de cada sistema.

Referências Bibliográficas

[1] World Health Organization, “Combined or multiple exposure to health stressors in indoor built environments: An evidence-based review prepared for the who training workshop “multiple environmental exposures and risks”,” World Health Organization, Bonn, Germany, Tech. Rep., 2014.

[2] A. Giron, K. Marinho, L. Drummond, S. Rocha, L. Galeno, R. Gonçalves, and G. Travassos, “Sistema de software iot para monitoramento de biossegurança de instalações,” in *Anais Estendidos do XXV Simpósio Brasileiro de Computação Aplicada à Saúde*. Porto Alegre, RS, Brasil: SBC, 2025, pp. 229–234. [Online]. Available: https://sol.sbc.org.br/index.php/sbcas_estendido/article/view/35613

[3] J. Kelly, K. Campbell, E. Gong, and P. Scuffham, “The internet of things: Impact and implications for health care delivery,” *J Med Internet Res*, vol. 22, no. 11, p. e20135, Nov 2020.

[4] V. Genésio, L. Rodriguez, and R. Braga, “Monipaep: Um sistema para monitoramento de sintomas de doenças endêmicas,” in *Anais Estendidos do XXIV Simpósio Brasileiro de Computação Aplicada à Saúde*. Porto Alegre, RS, Brasil: SBC, 2024, pp. 127–132. [Online]. Available: https://sol.sbc.org.br/index.php/sbcas_estendido/article/view/28716

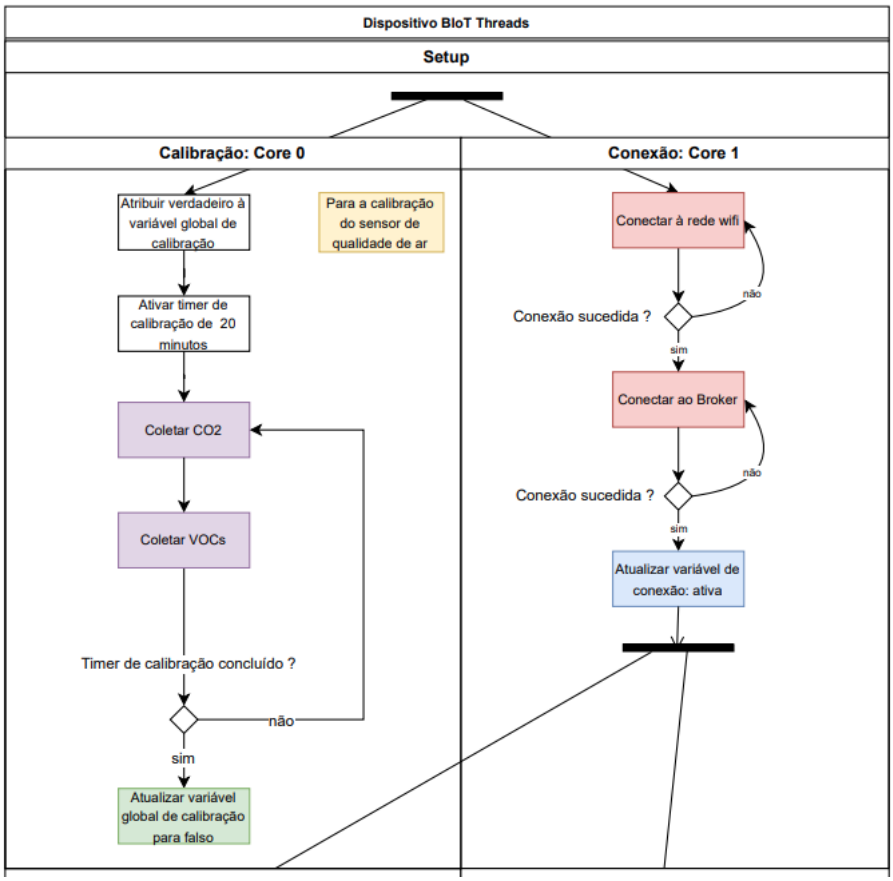
[5] A. B. d. Oliveira *et al.*, *Hospitais seguros e resilientes: Desafios e Estratégias de Preparação e Resposta a Emergências e Desastres*, 1st ed. Curitiba, PR, Brazil: Editora CRV, 2024.

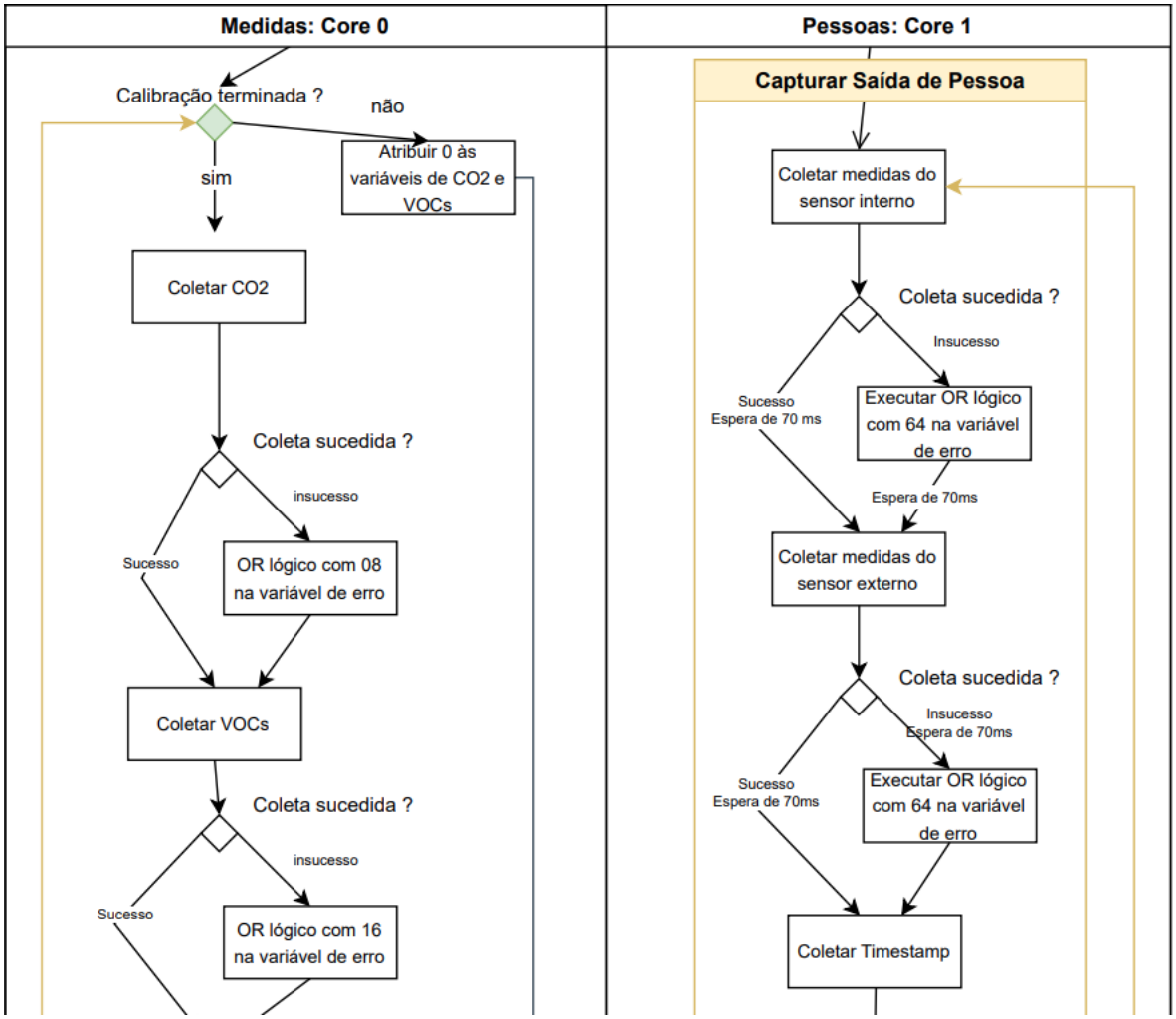
[6] UFRJ, “Guia de ações de biossegurança para resposta à pandemia pela covid-19 no Âmbito da ufrj,” Universidade Federal do Rio de Janeiro, Rio de Janeiro, Relatório técnico, 10 2020, disponível em: https://portal.nce.ufrj.br/images/documentos/retornopresencial/guia-de-biosseguranca-ufrj.pdf. Acesso em: 10/07/24.

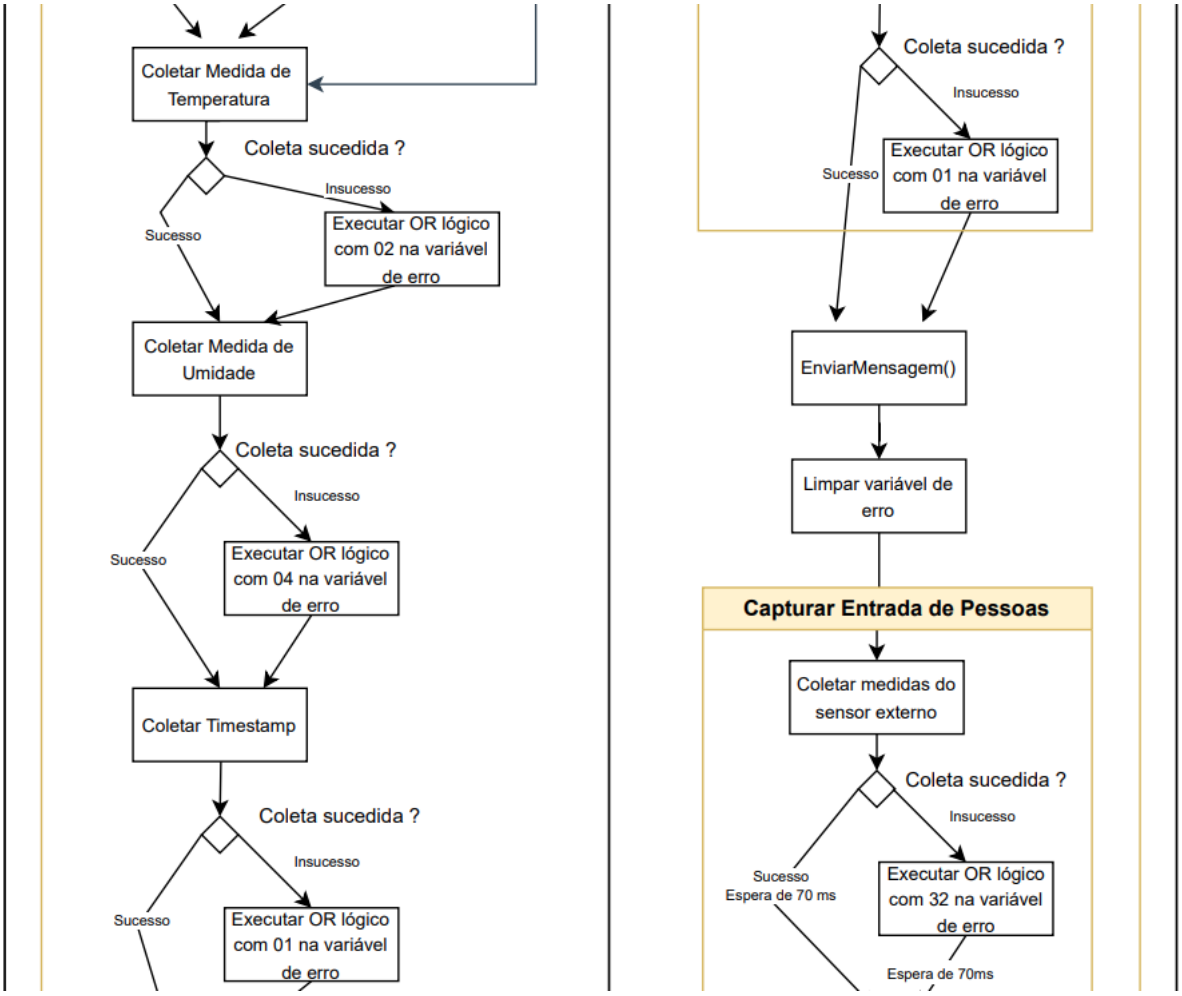
[7] N. Rios, R. Spínola, and G. H. Travassos, “Exploring technical debt on iot software projects,” in *Proceedings of the XXI Brazilian Symposium on Software Quality (SBQS ’22)*, ACM. Curitiba, Brazil: Association for Computing Machinery, 2022, p. 10. [Online]. Available: https://doi.org/10.1145/3571473.3571495

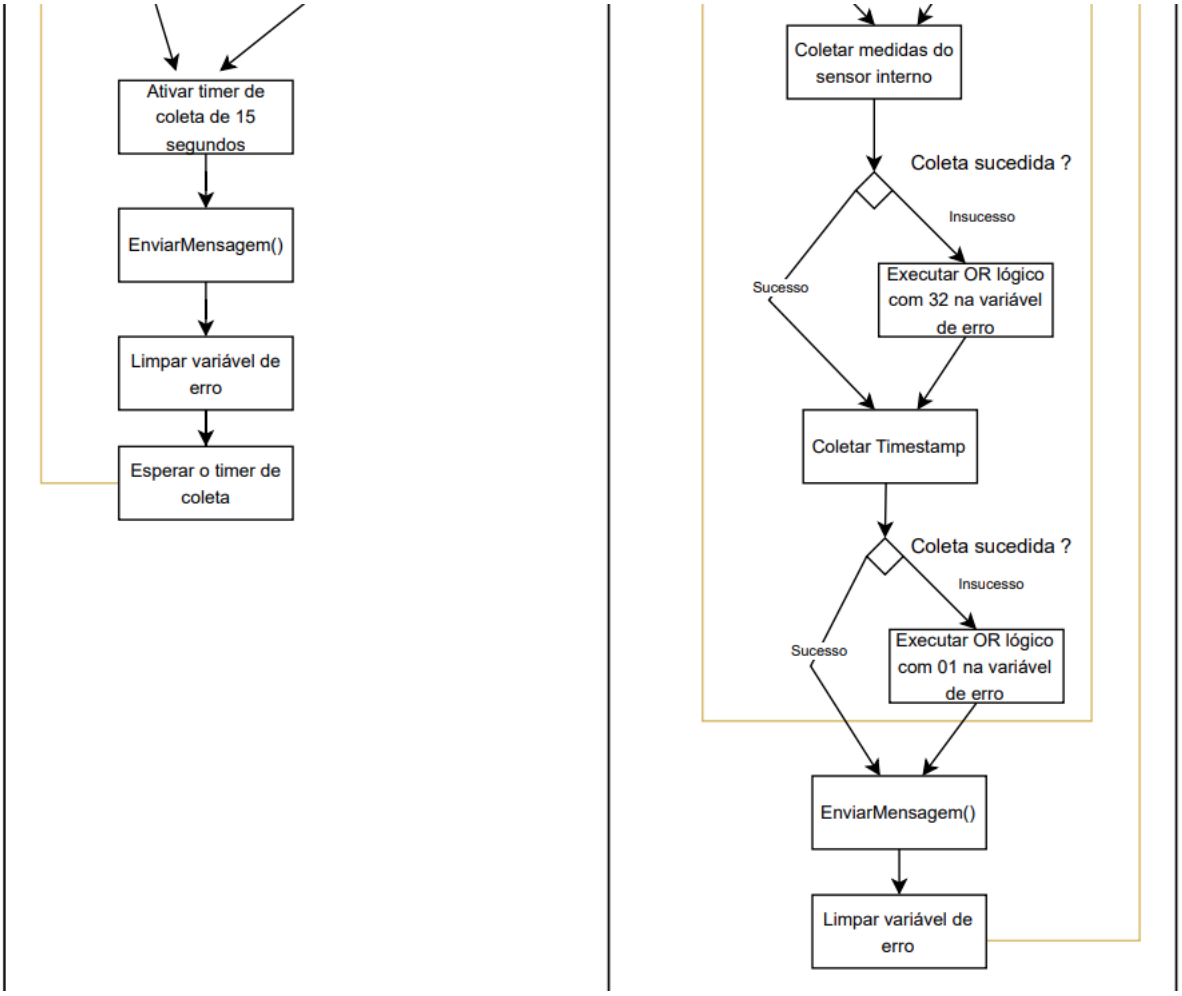
Apêndices

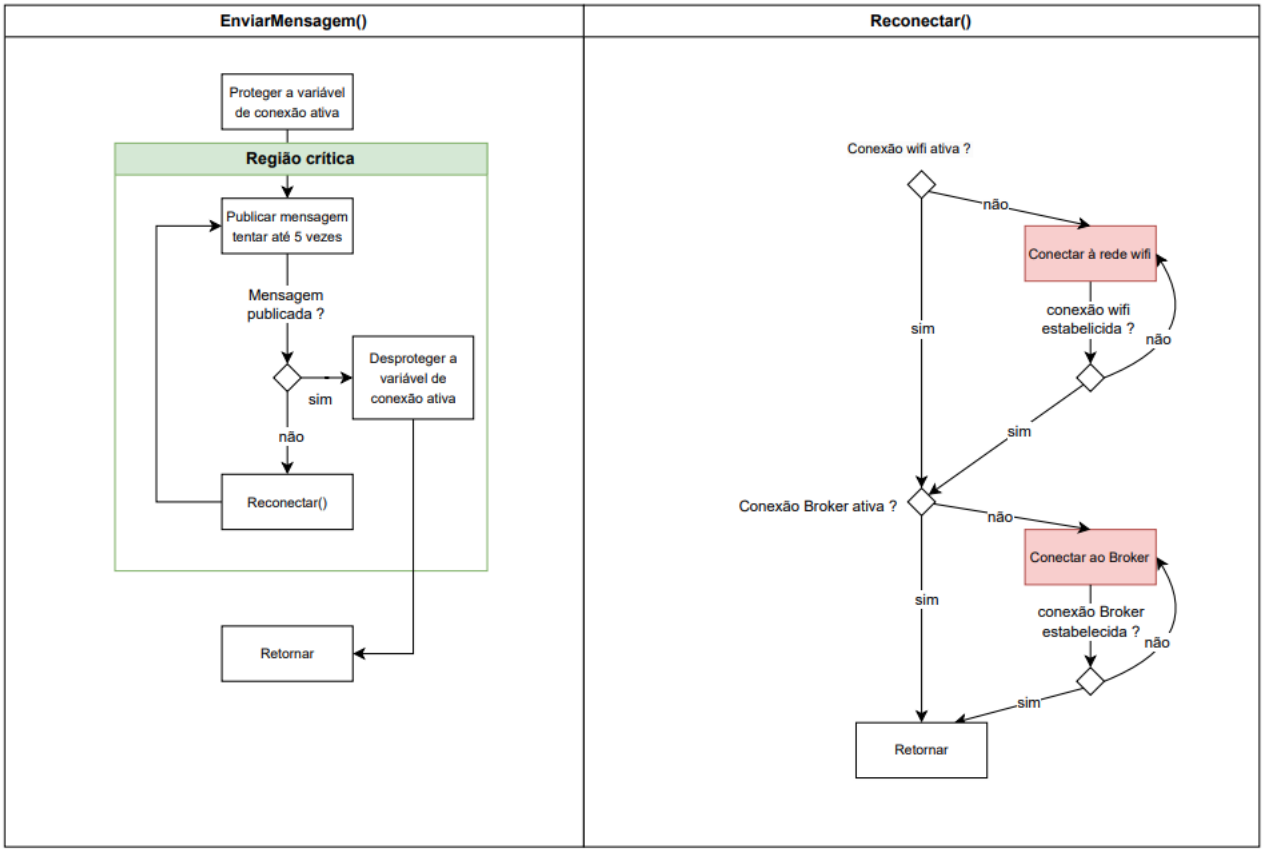
Apêndice A - Diagrama utilizado para reestruturação do código do dispositivo




Medidas: Core 0
Pessoas: Core 1
Calibração terminada ?
não
sim
Atribuir 0 às variáveis de CO2 e VOCs
Coletar CO2
Coleta sucedida ?
insucesso
Sucesso
OR lógico com 08 na variável de erro
Coletar VOCs
Coleta sucedida ?
insucesso
Sucesso
OR lógico com 16 na variável de erro
Capturar Saída de Pessoa
Coletar medidas do sensor interno
Coleta sucedida ?
Insucesso
Sucesso
Espera de 70 ms
Executar OR lógico com 64 na variável de erro
Espera de 70ms
Coletar medidas do sensor externo
Coleta sucedida ?
Insucesso
Espera de 70ms
Sucesso
Espera de 70ms
Executar OR lógico com 64 na variável de erro
Coletar Timestamp


Coletar Medida de Temperatura
Coleta sucedida ?
Insucesso
Sucesso
Executar OR lógico com 02 na variável de erro
Coletar Medida de Umidade
Coleta sucedida ?
Insucesso
Sucesso
Executar OR lógico com 04 na variável de erro
Coletar Timestamp
Coleta sucedida ?
Insucesso
Sucesso
Executar OR lógico com 01 na variável de erro
Coleta sucedida ?
Insucesso
Sucesso
Executar OR lógico com 01 na variável de erro
EnviarMensagem()
Limpar variável de erro
Capturar Entrada de Pessoas
Coletar medidas do sensor externo
Coleta sucedida ?
Insucesso
Sucesso
Espera de 70 ms
Executar OR lógico com 32 na variável de erro
Espera de 70ms


Ativar timer de coleta de 15 segundos
EnviarMensagem()
Limpar variável de erro
Esperar o timer de coleta
Coletar medidas do sensor interno
Coleta sucedida ?
Insucesso
Sucesso
Executar OR lógico com 32 na variável de erro
Coletar Timestamp
Coleta sucedida ?
Insucesso
Sucesso
Executar OR lógico com 01 na variável de erro
EnviarMensagem()
Limpar variável de erro


EnviarMensagem()
Proteger a variável de conexão ativa
Região crítica
Publicar mensagem tentar até 5 vezes
Mensagem publicada ?
sim
Desproteger a variável de conexão ativa
não
Reconectar()
Retornar
Reconectar()
Conexão wifi ativa ?
não
Conectar à rede wifi
conexão wifi estabelicida ?
não
sim
sim
Conexão Broker ativa ?
não
Conectar ao Broker
conexão Broker estabelecida ?
não
sim
sim
Retornar